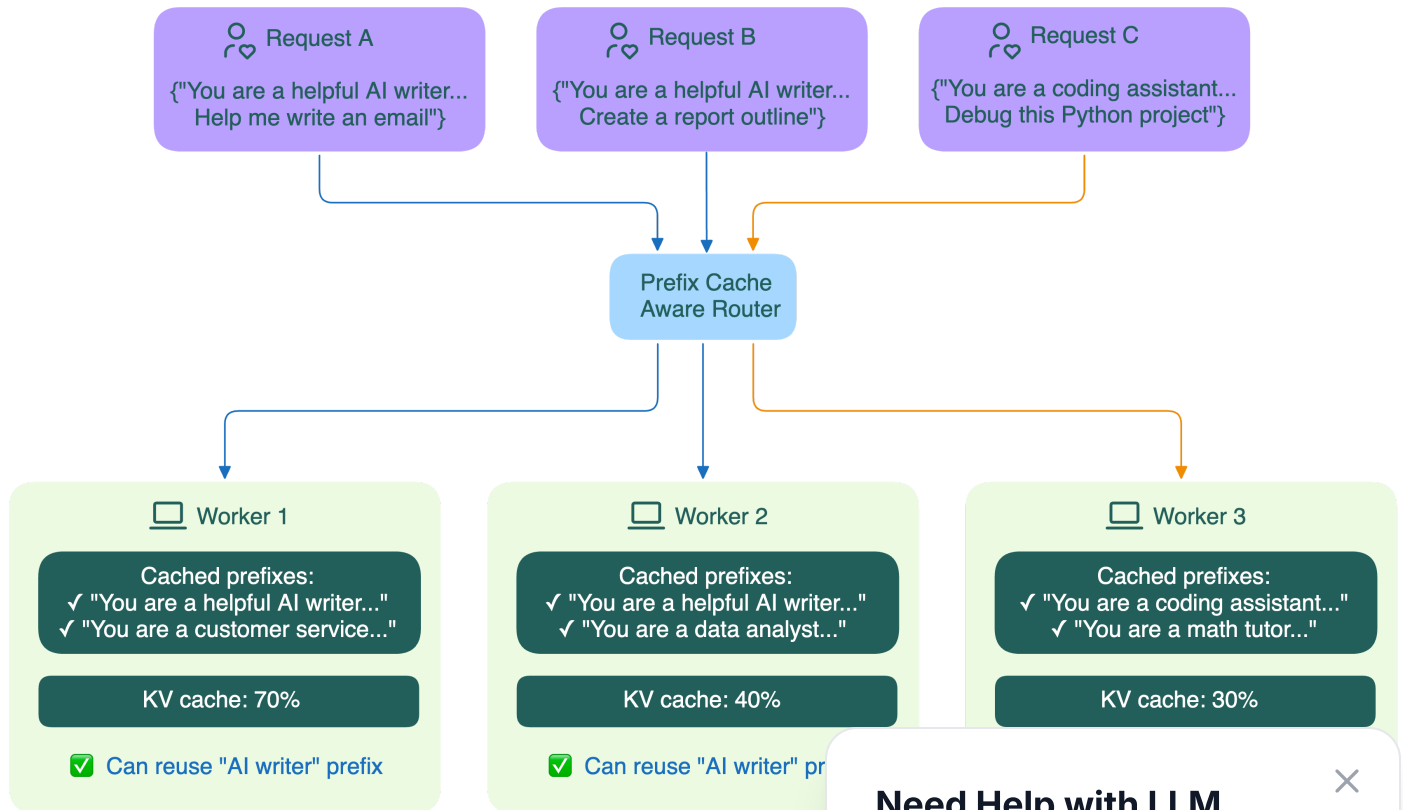


Prefix-aware routing

In practice, applying **prefix caching** in a distributed way still has challenges. For example:

- How can a new request be routed to the worker that already has the right prefix cached?
- How does the router know what's in each worker's cache?



Different open-source projects are exploring their own solutions to these challenges:

- **Worker-reported prefix status**

Dynamo has workers actively report which prefixes are cached, and uses this real-time data to make smart routing decisions

Need Help with LLM Inference?

We help top AI teams build high-performance LLM inference systems - fast, portable, no hardware lock-in.

[Request a Demo →](#)



BentoML is now part of Modular. [Read the announcement.](#)

SGLang maintains an approximate radix tree for each worker based on past requests. This helps the router predict which worker is most likely to have the needed prefix, without constant updates from the workers.

- **Hybrid efforts**

- The Gateway API Inference Extension project is **exploring multiple strategies to implement a routing algorithm on EPP**:
 - **Prefix affinity consistent hashing**: Group requests with similar prefixes to the same worker.
 - **Approximate prefix cache on the router**: Let the router maintain an approximate lookup cache of the prefix caches on all the backend servers.
 - **Accurate prefix cache on the router**: Gather KV cache information reported by model servers.
- The **llm-d** project uses a component called Inference Scheduler to implement filtering and scoring algorithms, and makes routing decisions based on a combination of factors like cache availability, prefill/decode status, SLA and load.

Need Help with LLM Inference?

We help top AI teams build high-performance LLM inference systems
- fast, portable, no hardware lock-in.

[Request a Demo →](#)